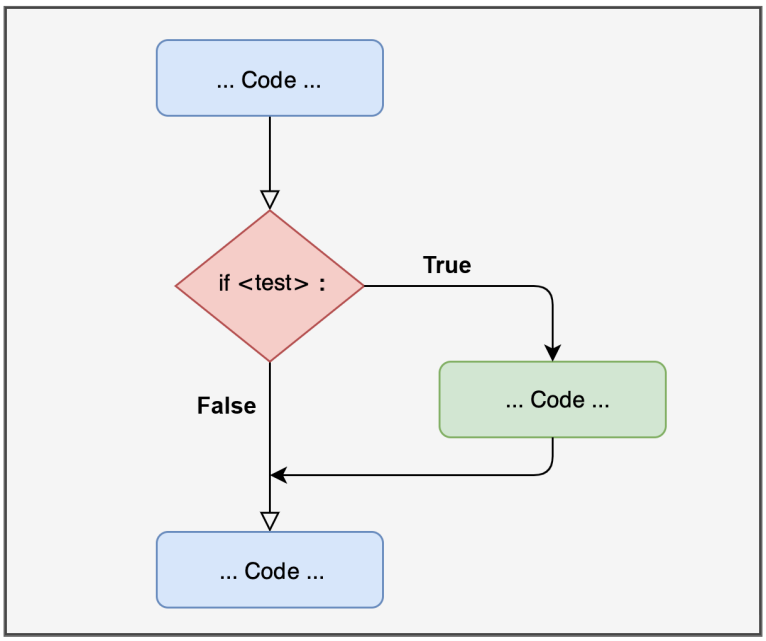
Programming: Conditionals, Loops, Custom Functions

Enrico Toffalini

`if` statement

automatically make decisions based on boolean value of a condition



Conditional Programming

The logic closely resembles that in R, except that Python uses indentation (**not** curly or round brackets) to define blocks of code

---

`if` statement

Performs an action *only if* a condition is met:

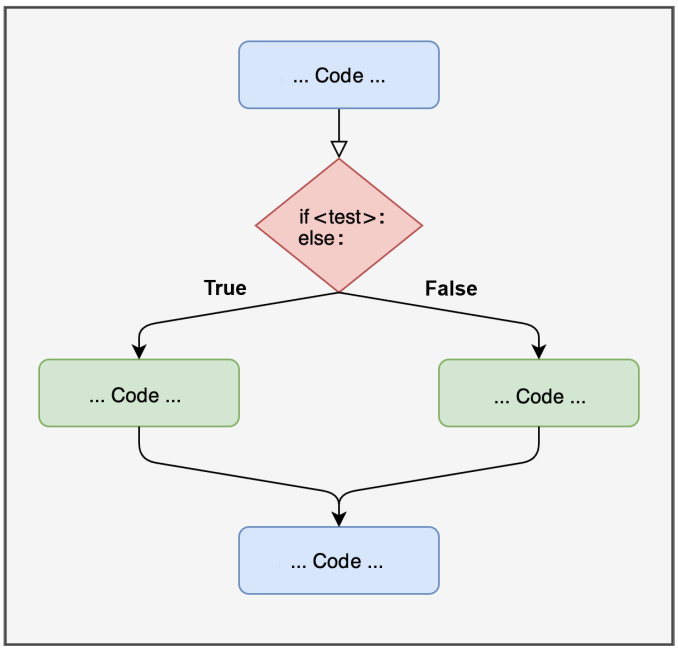
```
age = 20

if age >= 18:
    print("Adult")
```

```
Adult
```

`if...else` statement

Sometimes you need to perform *alternative, mutually-exclusive* actions:



`if...else` statement

```
age = 15

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

```
Minor
```

Note that **indentation** is really important!

```
age = 15

if age >= 18:
    print("Adult")
else:
print("Minor")
```

```
expected an indented block after 'else'
statement on line 5 (<string>, line 6)
```

`if...elif...else` statement

When you need to evaluate more than just two alternative conditions, you can use sort of *nested conditional statements* with with `if...elif...else`

```
age = 10

if age >= 18:
    print("Adult")
elif age >= 13:
    print("Adolescent")
elif age >= 2:
    print("Child")
else:
    print("Infant")
```

```
Child
```

Example: Preplanned Analysis

Example of **automated decision** in a hypothetical **pre-registered analysis pipeline**:

```
import numpy as np
import scipy.stats as st

x1 = np.random.normal(0, 1, size=30)
x2 = np.random.normal(0.5, 1, size=30)

tt = st.ttest_ind(x1, x2)
pval = tt.pvalue.round(4)
print(pval)
```

```
0.0121
```

```
if pval < 0.05:
    print("Significant result: proceeding with f
    # Here you could perform other analyses afte
else:
    print("No significant result: reporting prel
```

```
Significant result: proceeding with follow-up
analysis
```

Vectorized and Nested conditions

All previous examples evaluated **a single statement** that may be `True` or `False`. However, you often want to apply this operation to an entire vector

```
agesVector = np.array([2, 28, 15, 11, 4

if agesVector >= 18:
    print("Adult")
else:
    print("Minor")
```

```
ValueError: The truth value of an array with
more than one element is ambiguous. Use
a.any() or a.all()
```

the error message suggests that I might use `np.any(agesVector >= 18)` *or* `np.all(agesVector >= 18)`*, but this is not what I want! What I want is actually an* `if...else` *that evaluates across a whole vector of* `True`*s and* `False`*s (which should be like the* `ifelse()` *in R)*

Vectorized and Nested conditions

You could use ***list comprehension*** (more on this later!), but it's a bit **painful to write**, **less readable**, and **slower** (for large data)…

```
agesVector = np.array([2, 28, 15, 11, 4

["adult" if age >= 18 else "minor" for
```

```
['minor', 'adult', 'minor', 'minor', 'minor',
'adult', 'minor', 'adult', 'minor', 'minor']
```

Vectorized and Nested conditions with `np.where()` and `np.select()`

```
agesVector = np.array([2, 28, 15, 11, 4

np.where(agesVector >= 18, "Adult", "Mi
```

```
array(['Minor', 'Adult', 'Minor', 'Minor',
'Minor', 'Adult', 'Minor',
       'Adult', 'Minor', 'Minor'],
dtype='<U5')
```

↳ *manages one single condition, similar to `ifelse()` in R*

```
conditions = [agesVector >= 18, agesVec
                                agesVector
choices = ["Adult", "Adolescent", "Chil

np.select(conditions, choices, default=
```

```
array(['Child', 'Adult', 'Adolescent',
'Child', 'Child', 'Adult',
       'Infant', 'Adult', 'Adolescent',
'Child'], dtype='<U10')
```

↳ *manages multiple nested conditions; no direct equivalent in R, maybe `dplyr::case_when()`*

Loops in Python

Looping in Python is used to repeat actions: `for` and `while` are most common

`for` loop basics

```
for i in range
    print(i)
```

```
0
1
2
3
4
```

```
for i in range
    print(i**2
```

```
0
1
4
9
16
```

Time-based iteration

```
import time

for i in range
```

```
                print(time
                time.sleep
```

```
1748966362.699327
1748966363.6996806
1748966364.7001276
1748966365.7013485
1748966366.7020855
```

Monte Carlo Simulation 😀

Repeat a data simulation to estimate the *standard error of the mean*:

```
import numpy as np

N = 30
niter = 10
np.random.seed(0) # set seed for reproducibilit
results = np.empty(niter) # initialize empty ve

for i in range(niter):
    x = np.random.normal(size=N)
    results[i] = x.mean()

print( results.round(4) )
```

```
[ 0.4429 -0.2895 -0.1337  0.5108  0.0965 -0.0672
-0.1006 -0.0776 -0.304
  0.1978]
```

```
print( np.std(results).round(4) )
```

```
0.267
```

Monte Carlo Simulation 😀

```
# STEP 1: RUN SIMU

import numpy as np

N = 30
niter = 10000

np.random.seed(0)
results = np.empty

for i in range(nit
    x = np.random.
    results[i] = x

# STEP 2: ESTIMATE

print( np.std(resu
```

```
0.1814
```

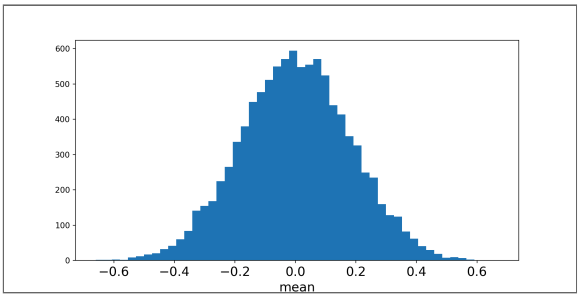
```
# STEP 3: PLOT RES

import matplotlib.

plt.hist(results,

plt.xticks(fontsiz
plt.xlabel("mean",

plt.show()
```


600
500
400
300
200
100
0
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
mean

Iterating over Elements

Iterating over a sequence of integers (e.g., "`i in range(niter)`" is a common practice, however you could also iterate directly over the elements of a *List* or other data structures

```
words = ["this", "is", "a", "vector", "of", "str

for w in words:
    print(w.upper()*4)
```

```
THISTHISTHISTHIS
ISISISIS
AAAA
VECTORVECTORVECTORVECTOR
OFOFOFOF
STRINGSSTRINGSSTRINGSSTRINGS
```

List comprehension is another, compact type of *for* loop over list elements:

```
[w.upper()*2 for w in words]
```

```
['THISTHIS', 'ISIS', 'AA', 'VECTORVECTOR', 'OFOF',
'STRINGSSTRINGS']
```

`while` loop

The `while` loop is another classical type of iterative structure. It is useful when the precise number of iterations is unknown *a priori*, and depends on a condition becoming `True`

```
amount = 1000
month = 0
interest_rate = 0.001

while amount < 1500:
    month += 1
    amount += amount * interest_rate

print(month)
```

```
406
```

➜ *it takes 406 months to reach an amount of €\(1,500\) when starting with an amount of €\(1,000\) with a 0.1% monthly interest rate*

`break` in loops

The `break` command allows to interrupt any loop based on a condition

```
import time
import scipy.stats as st

i = 0
pval = 1
Start = time.time()
while pval >= 0.001: # go on until p < 0.001
  i += 1
  x1 = np.random.normal(0,1,size=30)
  x2 = np.random.normal(0,1,size=30)
  tt = st.ttest_ind(x1, x2)
  pval = tt.pvalue
  Now = time.time()
  if Now - Start > 10:
    break             # however, stop if overall
print([i, pval.round(4)])
```

```
[1045, np.float64(0.0004)]
```

Other iteration: `for` with `zip()`

`zip()` pairs elements across multiple sequences while iterating them

```
numbers =


for n in r
    print(r
```

```
25
100
100
4
49
```

```
numbers = [5, 10,
exponents = [2, 1

for n, e in zip(nu
    print(n**e)
```

```
25
10
100000
8
7
```

```
[n**e for n, e in zip(numbers, exponent
```

```
[25, 10, 100000, 8, 7]
```

but note that the latter could be obtained much more easily with `numpy` vectorized operations: ***`np.array(numbers) ** np.array(exponents)`***

Other iteration: `for` with `zip()`

`zip()` pairs elements across multiple sequences while iterating them

```
teacher = ["Pastore", "Granziol", "Fera
course = ["CurrentIssues", "BasicsInfer
hours = [10, 20, 20, 5]

for t, c, h in zip(teacher, course, hou
    print(f"{t} teaches {c} which has {h
```

```
Pastore teaches CurrentIssues which has 10
hours
Granziol teaches BasicsInference which has 20
hours
Feraco teaches SEM which has 20 hours
Altoe teaches Outliers which has 5 hours
```

Other iteration: `for` with `zip()`

`zip()` is very convenient, but if really you don't like it, you could do the same task using the classical numerical iterator `i`

```
teacher = ["Pastore", "Granziol", "Fera
course = ["CurrentIssues", "BasicsInfer
hours = [10, 20, 20, 5]

for i in range(len(teacher)):
    print(f"{teacher[i]} teaches {course
```

```
Pastore teaches CurrentIssues which has 10
hours
Granziol teaches BasicsInference which has 20
hours
Feraco teaches SEM which has 20 hours
Altoe teaches Outliers which has 5 hours
```

Other iteration: `map()`

`map()` applies a specific function to each item in a sequence:

```
result = map(len, ["apple", "banana", [

list(result)
```

```
[5, 6, 3, 10]
```

in `map()`, you need to use `list(...)` to actually generate the result, otherwise a non-evaluated “lazy” `map` object is obtained

`zip()` and `map()` are about equivalent to `lapply()`/`sapply()` in R

Custom Functions

Custom functions are widely used in Python for efficiently reusing chunks of code. *Define* your functions with **def**; the logic is very similar as in R:

```
def zScore(vect):
    vect = np.array(vect)
    mu = np.mean(vect)
    sigma = np.std(vect)
    return ((vect - mu) / sigma)

a = [10, 14, 7.6, 18, 22, 50, 0.5]
b = [700, 131, 215, 133.2, 190, 4100, 108.9]
c = [-4.2, -10.2, 2, -15]

zScore(a).round(3)
```

```
array([-0.503, -0.233, -0.665,  0.038,  0.308,
2.201, -1.145])
```

```
zScore(b).round(3)
```

```
array([-0.071, -0.489, -0.427, -0.487, -0.446,
2.425, -0.505])
```

```
zScore(c).round(3)
```

```
array([ 0.415, -0.525,  1.386, -1.277])
```

Custom Functions with `def`

Let's elaborate the custom `zScore` function a little bit, adding another arguments that allows us to specify whether we want to ignore *missing values*:

```
def zScore(vect, naIgnore=False):
    vect = np.array(vect)
    if naIgnore==True:
        mu = np.nanmean(vect)
        sigma = np.nanstd(vect)
    else:
        mu = np.mean(vect)
        sigma = np.std(vect)
    return ((vect - mu) / sigma)


myVector = np.array([10, 14, 7.6, np.nan, 18, 2
zScore(myVector).round(2)
```

```
array([nan, nan, nan, nan, nan, nan, nan, nan, nan,
nan, nan])
```

```
zScore(myVector, naIgnore=True).round(2)
```

```
array([-0.32, -0.04, -0.49,   nan,  0.25,  0.53,  2.5
, -0.98,   nan,
       -0.92, -0.53])
```

Supercompact Custom Functions with `lambda`

`lambda` command allows you to define a function in a single line of code without `def` or `return`; it may be useful for quick transformation, but of course does not allow any complex "logic" / statement

```
myVector = np.array([10, 14, 7.6, 18, 2

zScore = lambda x: (x - x.mean()) / x.s

zScore(myVector)
```

```
array([-0.31638231, -0.03515359, -0.48511954,
0.24607513,  0.52730384,
       2.49590486, -0.98430051, -0.92102405,
-0.52730384])
```

Conditional programming: Python vs R

Task	Python	R
basic if	`if cond:`	`if(cond){ }`
if … else	`if cond:` `else:`	`if(cond){` `} else { }`

Multiple conditions	`if cond1:`
	`elif cond2:`
	`else:`
		`if(cond1){`
		`} else if(cond2){`
		`} else { }`

Block delimiter	*indentation*	`{ }`
"*not*" elementwise	`~cond`	`!cond`
Multiple checks	`(a > 1) & (b < 5)`	`(a > 1) & (b < 5)`
Vectorized condition	`np.where(conds, ifT, ifF)`	`ifelse(conds, ifT, ifF)`
Multiple/nested vectorized conditions	`np.select([...], [...])`	`dplyr::case_when()`

Loops and Functions: Python vs R

Task	Python	R
Loop over integers	`for i in range(n):`	`for(i in 1:n) { }`
Loop over elements	`for a in A:`	`for(a in A){ }`
While loop	`while cond:`	`while(cond){ }`
Block delimiter	*indentation*	`{ }`
Break loop	`break`	`break`
Apply function (list)	`list(map(func, A))`	`lapply(A, func)`
Multilist iteration	`for a, b in zip(A, B):`	`mapply(FUN, A, B)`
List comprehension	`[func(a) for a in A]`	`lapply(...)`

Function	`def myFunc(a):`
	`  ...`
	`  return ...`
		`myFunc = function(a){`
		`...`
		`return(...)`
		`}`

Supercompact function	`lambda a: a + 1`	`function(a) a + 1`